C#基础（3）

1 协变和逆变

1. 协变指能够使用比原始指定的派生类型的派生程度更大（更具体的）的类型。
2. 逆变指能够使用比原始指定的派生类型的派生程度更小（不太具体的）的类型。

2 委托

定义：存有对某些方法的引用的一种**引用类型**变量，可实现其它语言（C++）使用函数指针进行寻址的方案。 委托实例封装一个调用列表，它是一个或多个方法的列表，其中每个方法都称为可调用实体。从程序角度可以把委托视作用来执行函数的一个指针。

定义、声明、实例化、使用：

```
delegate double testDelegate(double x); // 定义，可引用任何一个带有单一double形参，返回double类型变量的方法
class Base
{
    public double fun(double x)
    {
        return x * Math.PI;
    }

}
class Program
{

    static void Main(String[] args)
    {
        Base b = new Base();
        testDelegate td = b.fun; // 委托推断
        // testDelegate td = new testDelegate(b.fun); // 普通实例化
        Console.WriteLine(td(3)); // 使用，输出：9.42477796076938
    }
}
```

具有相同参数列表和返回类型的两个委托类型被视为不同的委托类型。

```
delegate void D(int x);
delegate void DD(int x);
```

2.1 委托（C#）和函数指针(C++)

C++的函数指针不能像C#委托一样指向不同类的方法；不具有对象的状态性质；在使用上函数指针也不如委托灵活。

委托可以指向任意方法，包括不同类中的方法，只要符合委托签名。

```
delegate int testDelegate(int x, int y);
class ClassA
{
```

```
    public int Add(int x, int y)
    {
        return x + y;
    }
}
class ClassB
{
    public int Add(int x, int y)
    {
        return x + y + 1;
    }
}
class Program
{
    static int Multiple1(int x, int y)
    {
        return x * y;
    }
    static int Multiple2(int x, int y)
    {
        return x * y * 3;
    }
    static void Main(string[] args)
    {
        testDelegate td = Multiple1;
        Console.WriteLine(td(2, 3)); // 6
        td = Multiple2;
        Console.WriteLine(td(2, 3)); // 18

        ClassA a = new ClassA();
        ClassB b = new ClassB();
        td = a.Add;
        Console.WriteLine(td(2, 3)); // 5
        td = b.Add;
        Console.WriteLine(td(2, 3)); // 6
    }
}
```

C/C++的函数指针变量只是单纯的函数入口地址，并且指明了可接受的函数类型，因此不能指向不同类中的方法。

```
#include<iostream>
using namespace std;

int Multiple1(int x, int y) { return x * y; }
int Multiple2(int x, int y) { return x * y * 3; }

class ClassA {
public:
    ClassA() {}
    int Add(int x, int y) { return x + y; }
};
class ClassB {
public:
    ClassB() {}
```

```
    int Add(int x, int y) { return x + y + 1; }
};

typedef int(*FN)(int x, int y);
typedef int(ClassA::*FA)(int x, int y);
typedef int(ClassB::*FB)(int x, int y);

int main()
{
    FN fn = Multiple1;
    cout << fn(2, 3) << endl; // 6
    fn = Multiple2;
    cout << fn(2, 3) << endl; // 18
    fn = &ClassA::Add; // 不能将 "int (ClassA::*)(int x, int y)" 类型的值分配到 "FN" 类型的实体
    fn = &ClassB::Add; // 不能将 "int (ClassB::*)(int x, int y)" 类型的值分配到 "FN" 类型的实体

    ClassA a;
    FA fa = &ClassA::Add;
    cout << (a.*fa)(2, 3) << endl; // 5
    fa = Multiple1; // 不能将 "int (*)(int x, int y)" 类型的值分配到 "FA" 类型的实体
    fa = &ClassB::Add; // 不能将 "int (ClassB::*)(int x, int y)" 类型的值分配到 "FA" 类型的实体

    ClassB b;
    FB fb = &ClassB::Add;
    cout << (b.*fb)(2, 3) << endl; // 6
    fb = Multiple1; // 不能将 "int (*)(int x, int y)" 类型的值分配到 "FB" 类型的实体
    fb = &ClassA::Add; // 不能将 "int (ClassB::*)(int x, int y)" 类型的值分配到 "FB" 类型的实体
}
```

2.2 委托的多播

委托对象可以用`+`来合并（相同类型的委托），也可以用`-`来解除，**合并、执行时都按照先后次序，解除时顺序相反。**

```
delegate void D(int x);
class Base
{
    public void M1(int i)
    {
        Console.WriteLine("M1");
    }
    public void M2(int i)
    {
        Console.WriteLine("M2");
    }

}
class Test
{
```

```
    static void Main()
    {
        Base b = new Base();
        D d1 = new D(b.M1);     // M1
        D d2 = new D(b.M2);     // M2
        D d3 = d1 + d2;         // M1 + M2
        D d4 = d3 + d1;         // M1 + M2 + M1
        D d5 = d4 + d3;         // M1 + M2 + M1 + M1 + M2
        d5(0);
        Console.WriteLine();
        d5 -= d1;               // M1 + M2 + M1 + M2
        d5(0);
        Console.WriteLine();
        d5 -= d4;               // M2
        d5(0);
    }
}
```

使用GetInvocationList()方法来按顺序获取并执行委托对象调用列表中的方法。

```
delegate int D(int x);
class Base
{
    public int M1(int i)
    {
        return 1;
    }
    public int M2(int i)
    {
        return 2;
    }

}
class Test
{
    static void Main()
    {
        Base b = new Base();
        D d1 = new D(b.M1);
        D d2 = new D(b.M2);
        d1 += d2;
        Console.WriteLine(d1(0)); // 2
        foreach (D d in d1.GetInvocationList())
            Console.WriteLine(d.Invoke(0)); // 1\n2
    }
}
```

2.2.1 委托中的相等运算符

当且仅当两个相同的委托对象的调用列表项中每个条目都相等时，这两个委托对象才相等。

```
delegate void D(int x);
delegate void DD(int x);
class Base
```

```
{
    public void M1(int i)
    {
        Console.WriteLine("M1");
    }
    public void M2(int i)
    {
        Console.WriteLine("M2");
    }

}
class Test
{
    static void Main()
    {
        Base b = new Base();
        D d1 = new D(b.M1);
        D d2 = new D(b.M2);
        Console.WriteLine(d1 + d2 == d2 + d1); // False
        Console.WriteLine(d1 + d2 == new D(b.M1) + d2); // True
    }
}
```

3 泛型委托

3.1 Func

Func<T1,T2,T3,T4,T5,T6,T7,T8,T9,T10,T11,T12,T13,T14,T15,T16,TResult> ,其中T1-T16均为输入参数，TResult为委托的返回值类型。输入参数最少0个，代表该委托是无参的，最多16个。

显式使用Func委托。

```
class Program
{
    static void Main(String[] args)
    {
        Func<int, int, int> func = Add;
        Console.WriteLine(func(3, 2)); // 5
        func = Sub;
        Console.WriteLine(func(3, 2)); // 1
    }
    static int Add(int x, int y)
    {
        return x + y;
    }
    static int Sub(int x, int y)
    {
        return x - y;
    }
}
```

隐式使用Func委托。

```
class Program
```

```
{
    static void Main(String[] args)
    {
        Console.WriteLine(Add(3, 2, Sub)); // 1
    }
    static int Add(int x, int y, Func<int, int, int> func)
    {
        return func(x, y);
    }
    static int Sub(int x, int y)
    {
        return x - y;
    }
}
```

使用具有协变类型参数的委托，返回类型为派生类的方法可以分配给返回类型为基类的Func委托，此时 `func1()` 和 `func2()` 的返回值都是派生类类型。

```
class Base { }
class Derived : Base { }
class Program
{
    static void Main(String[] args)
    {
        Func<Derived> func1 = Test;
        Func<Base> func2 = Test;
        func1(); // Run Test
        func2(); // Run Test
    }
    static Derived Test()
    {
        Console.WriteLine("Run Test");
        return new Derived();
    }
}
```

3.2 Action

Action<T1,T2,T3,T4,T5,T6,T7,T8,T9,T10,T11,T12,T13,T14,T15,T16>。与Func区别在于没有返回值，适用于任何没有返回值的委托类型。

显式使用Action委托。

```
class Program
{
    static void Main(String[] args)
    {
        Action act = Print1;
        act(); // hello
        act = Print2;
        act(); // world
    }
    static void Print1()
    {
```

```
        Console.WriteLine("hello");
    }
    static void Print2()
    {
        Console.WriteLine("world");
    }
}
```

隐式使用Action委托。

```
class Program
{
    static void Main(String[] args)
    {
        Print1(Print2); // INS
    }
    static void Print1(Action act)
    {
        act();
    }
    static void Print2()
    {
        Console.WriteLine("INS");
    }
}
```

使用具有逆变类型参数的委托，**输入参数为基类的方法可以分配给输入参数为派生类的Action委托，此时`act1`只能传入基类类型的参数，`act2`只能传入派生类类型的参数。**

```
class Base { }
class Derived : Base { }
class Program
{
    static void Main(String[] args)
    {
        Base b = new Base();
        Derived d = new Derived();
        Action<Base> act1 = Test;
        Action<Derived> act2 = Test;
        act1(b); // Run Test
        act2(d); // Run Test
    }
    static void Test(Base b)
    {
        Console.WriteLine("Run Test");
    }
}
```

3.3 Predicate

封装返回值为bool类型的委托，可被Func代替。

4 事件

定义：事件建立在对委托的支持之上。事件是对象用于(向系统中的所有相关组件)广播已发生事情的一种方式。任何其他组件都可以订阅事件，并在事件引发时得到通知。

特性：

1. 事件拥有者确定何时引发事件；事件响应者确定对事件作出何种响应。
2. 一个事件可以有多个响应者。响应者可以处理来自多个拥有者的多个事件。
3. 没有事件响应者的事件永远也不会引发。
4. 事件不会主动发生，**事件拥有者内部逻辑触发后才能发生**
5. 当事件具有多个响应者时，引发该事件时会同步调用事件处理程器
6. 事件只能用`+=`和`-=`来订阅和取消订阅事件；**在声明事件的类中可以使用`=`来订阅事件，其它地方都不能直接用`=`来订阅事件**
7. **事件将委托对外界隐藏了，但是在声明事件的类中仍然可以使用委托**
8. 在 .NET 类库中，事件基于EventHandler委托和EventArgs基类

4.1 事件模型

组成：

1. 事件的拥有者（event source，对象）：事件的主体，事件消息的发送者
2. 事件（event，成员）
3. 事件的响应者（event subscriber，对象）：订阅了事件的对象，当一个事件发生时，被通知到的对象就是事件的响应者
4. 事件的处理器（event handler，成员）：事件响应者的方法成员
5. 事件订阅：把事件处理器与事件关联在一起，本质上是一种以委托类型为基础的约定

组合方式：

1. 事件拥有者和响应者是完全不同的两个对象：标准事件机制模型
2. 事件的拥有者和响应者是同一个：相当于一个对象拿自己的事件处理器去订阅和处理自己的事件
3. 事件拥有者是响应者的一个字段成员：window默认的事件订阅和处理结构，应用广泛
4. 事件响应者是拥有者的一个字段成员：不常用

以下示例展示了事件模型的组合1、2。组成如下：

1. 事件的拥有者：publisher
2. 事件：TestEvent
3. 事件的响应者：subscriber，Publisher类
4. 事件的处理器：Publisher类和Subscriber类中的Fun函数
5. 事件订阅：`=`，`+=`

```
class Publisher
{
```

```
    public delegate void TestDelegate();
    public event TestDelegate TestEvent;
    public Publisher()
    {
        TestEvent = Fun;
    }
    public void EventRun()
    {
        if(TestEvent != null) // 事件判空
        {
            TestEvent();
        }
        else
        {
            Console.WriteLine("Event Failed");
        }
    }
    public void Fun()
    {
        Console.WriteLine("Run Publisher.Fun()");
    }
}
class Subscriber
{
    public void Fun()
    {
        Console.WriteLine("Run Subscriber.Fun()");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Publisher publisher = new Publisher();
        Subscriber subscriber = new Subscriber();
        publisher.EventRun();
        Console.WriteLine();
        publisher.TestEvent += subscriber.Fun;
        publisher.EventRun();
    }
}
```

编程规范：

1. 不要提供public的事件成员变量，而是使用**事件访问器（类似属性访问器）**

```
private event TestDelegate TestEvent;
public event TestDelegate TEvent
{
    add { }
    remove { }
}
```

2. 事件的触发权限不能交给外界，应由事件拥有者的内部逻辑来控制触发

```
protected virtual void EventRun()
{
    if (TestEvent != null) // 事件判空
    {
        TestEvent();
    }
    else
    {
        Console.WriteLine("Event Failed");
    }
}
```

运行结果：

```
Run Publisher.Fun()

Run Publisher.Fun()
Run Subscriber.Fun()
```

4.2 委托和事件的区别

1. 委托可以在类内部和外部进行“调用”；事件只能在类内部“触发”
2. 委托常用匿名函数；事件要尽量避免使用匿名函数，因为匿名函数很难取消订阅
3. 委托可以有返回值；**事件没有返回值**
4. 委托是对方法的封装；事件是对委托的封装

4.3 EventHandler

委托EventHandler<TEventArgs>是一个**预定义**的委托，表示生成数据的事件处理程序方法。使用EventHandler<TEventArgs>的优点是，如果事件生成事件数据，则无需编写自己的自定义委托代码。只需提供事件数据对象的类型作为泛型参数。第一个参数sender是Object类型，表示事件拥有者；第二个参数派生自EventArgs类型，提供保存事件数据所需的任何字段或属性。

```
public delegate void EventHandler<TEventArgs>(object? sender, TEventArgs e);
```

优点：

1. 不必重复性的定义事件的原型委托（多种不同接受参数、返回值）
2. 搭配EventArgs，可传递多个信息；如果只传入某个固定参数，也可将<TEventArgs>换成<指定数据类型>；不搭配EventArgs可以用于处理不具有事件数据的事件

```
class StuArgs : EventArgs
{
    public string Name { get; set; }
    public string Number { get; set; }
    public StuArgs(string name, string number)
    {
        this.Name = name;
        this.Number = number;
    }
}
```

```
class Student
{
    public event EventHandler<StuArgs> handler;
    public Student()
    {
        handler += Print;
    }
    public void Print(object sender, StuArgs args)
    {
        Console.WriteLine($"姓名：{args.Name}，学号：{args.Number}");
    }
    public void callPrint(StuArgs args)
    {
        if(handler != null)
        {
            handler(this, args);
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        StuArgs stuargs = new StuArgs("张三", "33");
        Student stu = new Student();
        stu.callPrint(stuargs);
    }
}
```

4.4 内存泄漏